



5954014 - Engenharia de Software

Aula 10 - Modelagem de Software

Prof. Dr. Denis Martins

DCM | FFCLRP | USP

Tarefa Difícil

Interpretações divergentes

Diferentes pessoas interpretaram a necessidade de formas diferentes.

Perda de conhecimento

Conhecimento e decisões foram sendo perdidos ou modificados.

Visões isoladas

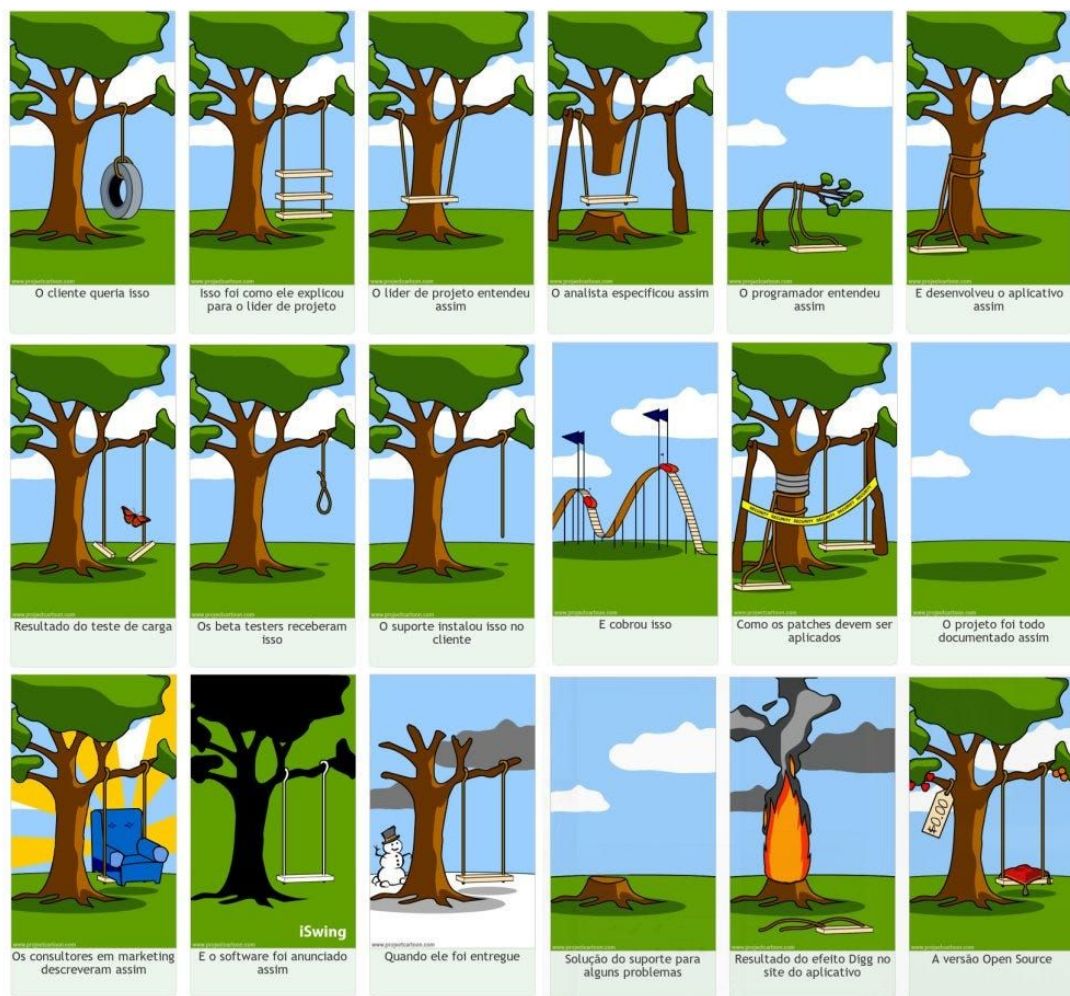
Cada etapa trabalhou com sua própria interpretação do problema.

Desvio do objetivo

O resultado final se afastou da necessidade original.

SOLUÇÃO DA ENGENHARIA DE REQUISITOS

Engenharia de Requisitos busca construir e manter um entendimento compartilhado entre stakeholders e equipe de desenvolvimento.

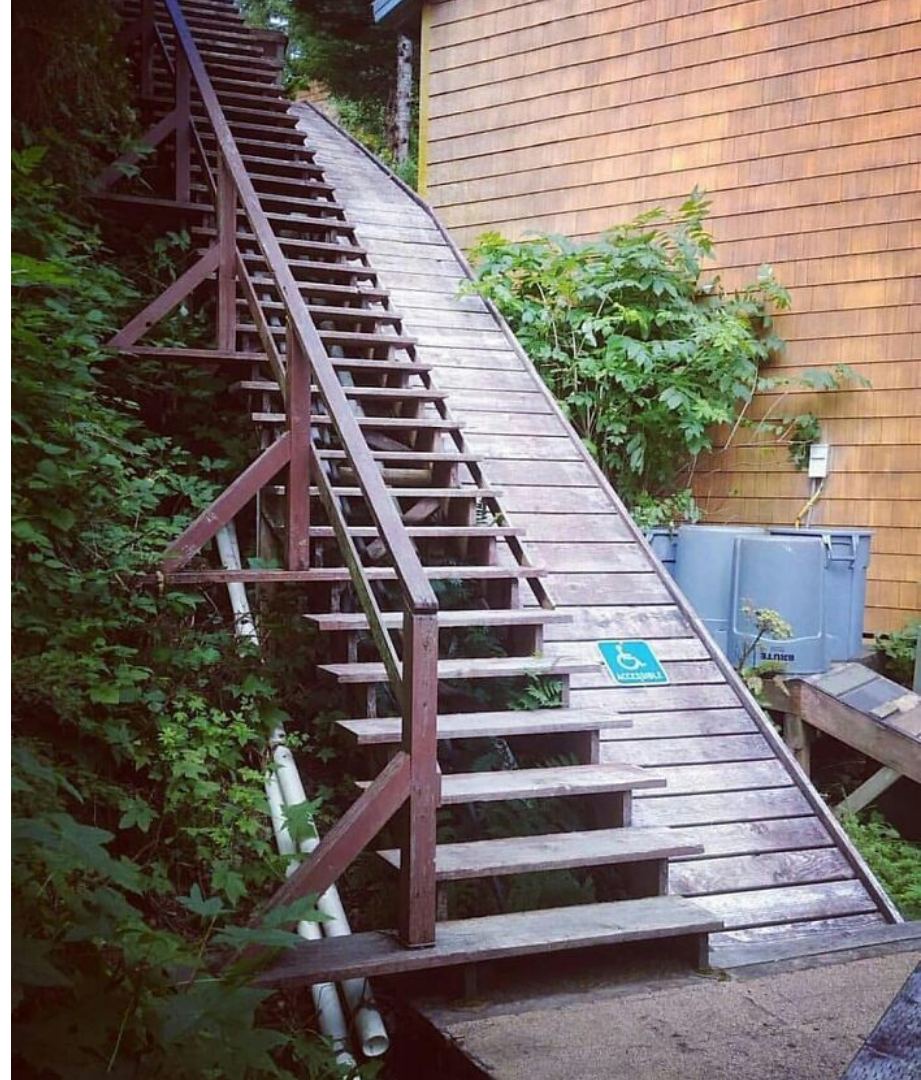


Frederick Brooks

The **hardest** single part
of building a
software system is
deciding precisely
what to build.

Fonte: https://en.wikipedia.org/wiki/Fred_Brooks





**Requisitos são suficientes para
começarmos a programar?**

Como aluno, quero reservar uma sala disponível para realizar uma atividade acadêmica.

Esse requisito é **suficiente** para começarmos a programar?

Requisitos indicam o que atender, não como projetar

Requisito Exemplo (formato de história de usuário)

Como aluno, quero reservar uma sala disponível para realizar uma atividade acadêmica.

Como passamos de “o que o sistema deve fazer” para uma representação que ajude a discutir e projetar a solução?”

Isso já é suficiente para projetarmos a solução?

- > O que é exatamente uma "reserva"?
- > Que elementos estruturais se relacionam?
- > Como o sistema verifica a disponibilidade?
- > Quem participa ativamente do processo?
- > O que acontece quando há necessidade de aprovação?

Níveis de Abstração em Engenharia de Software

INTENÇÃO

Requisitos

Dizem principalmente **o que e como precisamos** construir.

PONTE / TRANSIÇÃO

Modelos

Ajudam a **transitar** entre os requisitos e o código concreto.

EXECUÇÃO

Código

Mostra **concretamente** o que e como construímos.



Objetivos de Aprendizagem

01

Papel da Modelagem

Compreender o papel da modelagem como ponte entre requisitos e implementação.

02

Perspectivas e Diagramas

Distinguir diferentes perspectivas de modelagem e o propósito de cada diagrama.

03

Modelos UML Básicos

Interpretar e construir modelos UML básicos, especialmente casos de uso, sequência e atividades.

04

Transformação de Requisitos

Transformar requisitos textuais em modelos, justificando as principais decisões de modelagem.

Por que modelar software?

Modelos preenchem a lacuna entre os requisitos textuais abstratos e o código-fonte executável, sendo menos complexos que o código.

Principais Finalidades



Reduzir complexidade

Abstrair detalhes desnecessários para focar no essencial.



Comunicar

Facilitar o entendimento entre desenvolvedores e stakeholders.



Analisar

Compreender profundamente o domínio e os processos.



Projetar

Estruturar a solução antes da codificação.

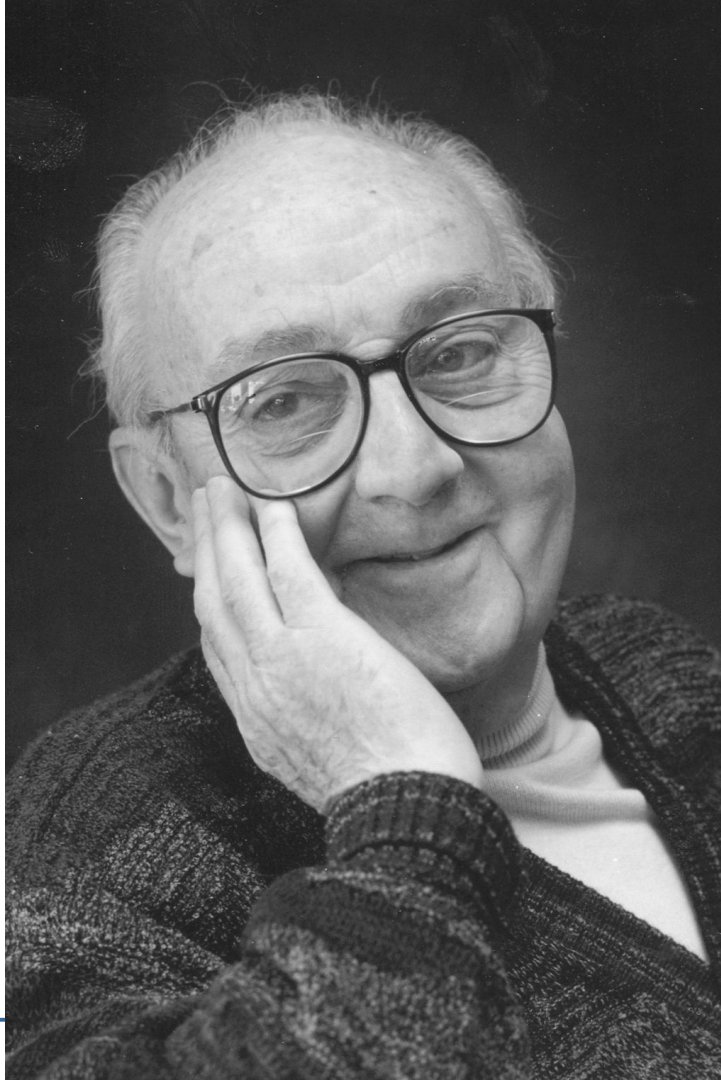
Regra Prática: Um modelo útil não precisa representar absolutamente tudo do sistema; ele precisa representar bem **aquilo que queremos discutir e resolver**.

George E. P. Box

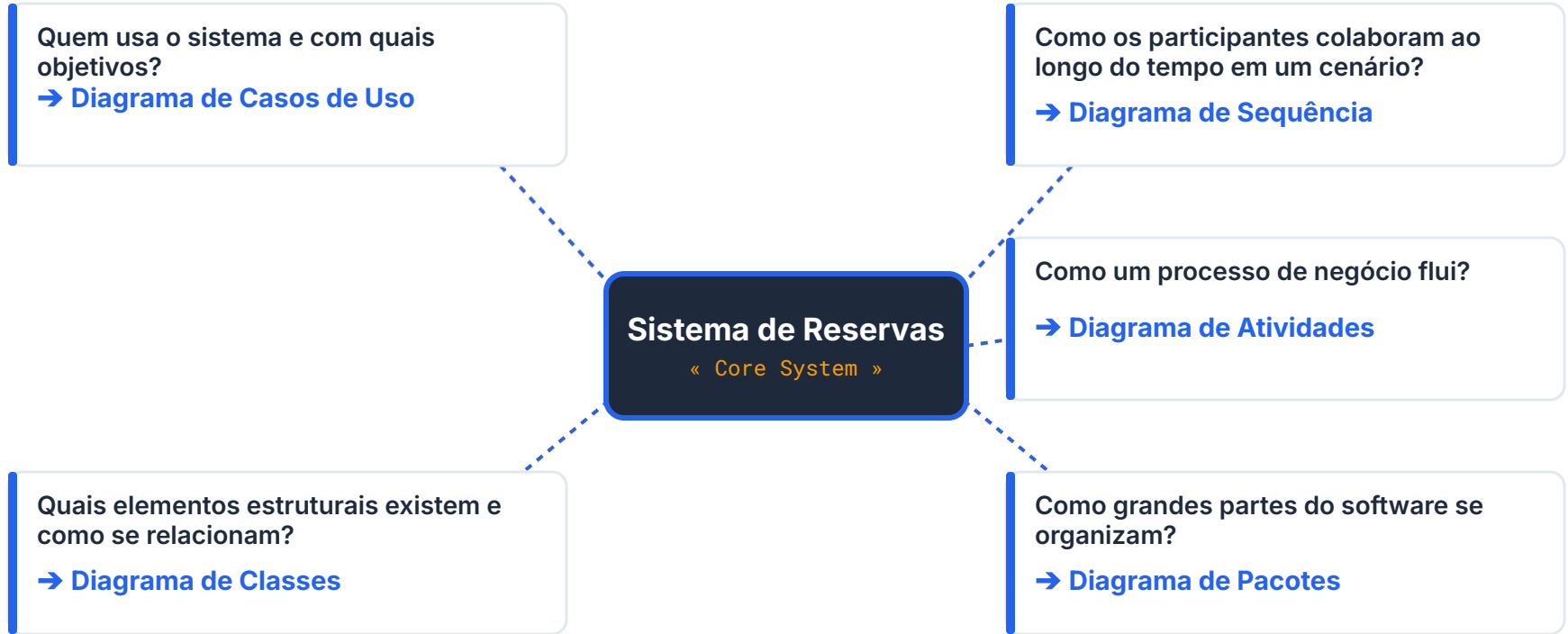
**All models are *wrong*, but
some models are *useful*.**

*So the question you need to ask is not "Is the model true?" (it never is) but "Is the model **good enough** for this particular application?"*

Fonte: https://en.wikipedia.org/wiki/George_E._P._Box



Um sistema, várias perspectivas



UML e Notação

UML como Blueprint

A Unified Modeling Language (UML) é a notação gráfica mais utilizada, mas possui abordagens diferentes:

- **Modelos exaustivos:** Cobertura detalhada e rigorosa de todo o sistema.
- **Documentação formal:** Foco em padronização e especificações completas.
- **Desenhos técnicos:** Projetos meticulosos para orientação direta de codificação.



UML como Sketch (Esboço)

Modelos seletivos, leves e informais.

O objetivo não é a completude, mas sim o apoio rápido à conversa e ao design.

Importante

Adotaremos a abordagem da **UML como sketch**, focando em comunicação ágil em vez de documentação pesada.

Diagrama de Casos de Uso: o sistema visto pelos atores

Ele atua como a principal ponte entre o levantamento de requisitos e a modelagem técnica, focando na interação externa.

1 Requisitos Textuais

Necessidades documentadas em linguagem natural.

2 Atores e Objetivos

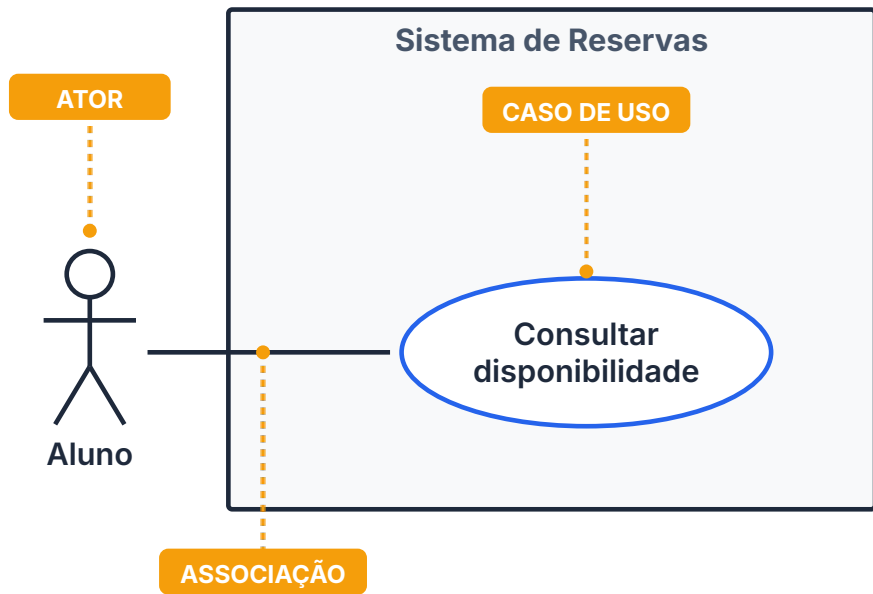
Identificação de quem interage e o que deseja alcançar.

3 Diagrama de Casos de Uso

Representação gráfica do escopo e das interações externas.

! O foco é manter a **perspectiva do usuário**, documentando o valor entregue sem detalhar a tecnologia ou a implementação interna.

Diagrama de Casos de Uso: o sistema visto pelos atores



Ator: Representa um papel desempenhado por um usuário ou sistema externo.

Caso de Uso (Elipse): Representa um objetivo ou funcionalidade entregue ao ator.

Limite do Sistema (Retângulo): Define a fronteira do software.

Associação (Linha): Conecta o ator ao caso de uso do qual ele participa.

Observações: A linha de associação não representa fluxo de dados nem ordem temporal.

Exemplo Mínimo: Aluno → [Consultar disponibilidade] dentro do Sistema de Reservas.

Casos de uso descrevem **O QUE** o sistema deve oferecer ao ator — não **COMO** será implementado.

Casos de Uso: Documentação complementa o Diagrama

DOCUMENTO TEXTUAL

Caso de Uso: Solicitar reserva

Ator: Aluno

Fluxo Principal:

1. Informar data.
2. Verificar disponibilidade.
3. Confirmar reserva.

Extensão:

2a. Se indisponível, sugerir horários.

DIAGRAMA GRÁFICO



- Visão sintética e visual do escopo.
- Mostra os Atores e seus Objetivos.
- Define o Limite do Sistema.
- Mapeia os Relacionamentos de participação.

Como descobrir atores e casos de uso?

1

Defina a fronteira

O que faz parte do sistema e o que é externo?

2

Identifique atores

Quem interage diretamente com o sistema?

3

Identifique objetivos

O que cada ator precisa realizar?

4

Revise o escopo

Garanta o nível correto de abstração.

Dica Prática: Prefira nomes no formato "**Verbo + Objeto**"

Exemplo: Solicitar reserva



Alerta (Evite): Nomes como "clicar botão", "abrir tela" ou "digitar campo".

Eles descrevem a interface gráfica, não os objetivos de negócio.

Diagrama de Casos de Uso: Exemplo

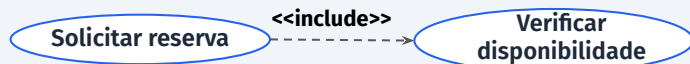


Diagrama de Casos de Uso: Exemplo

<<include>>

Comportamento obrigatoriamente reutilizado.

A execução do caso de uso base exige a execução do incluído.



Exemplo: Solicitar reserva sempre verifica a disponibilidade.

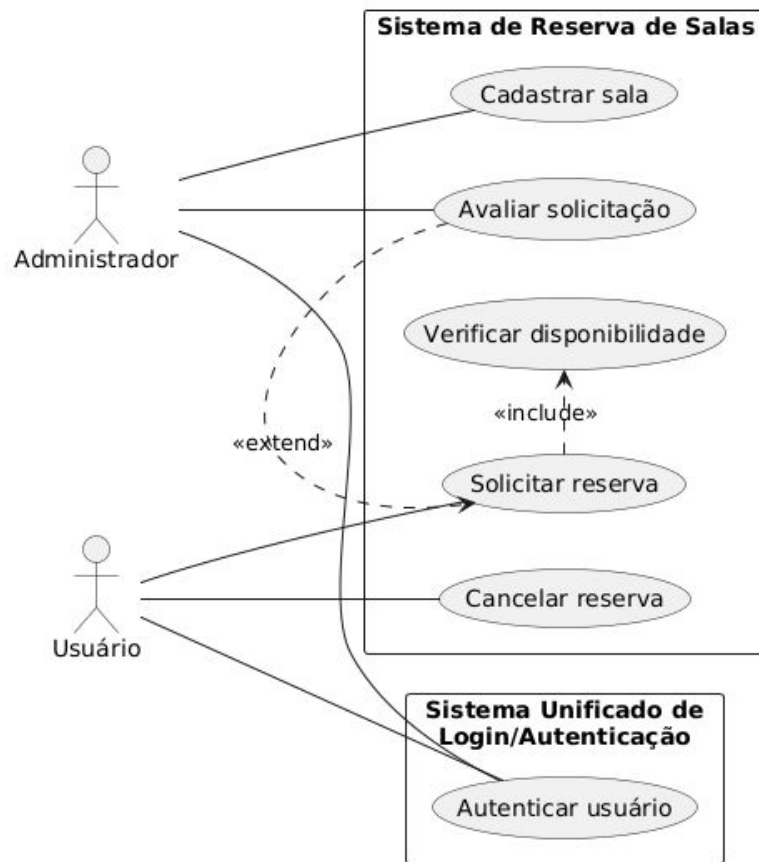
<<extend>>

Comportamento adicional ou condicional.

Ocorre apenas sob certas regras ou condições específicas.



Exemplo: Aprovação é solicitada apenas se regras especiais exigirem.



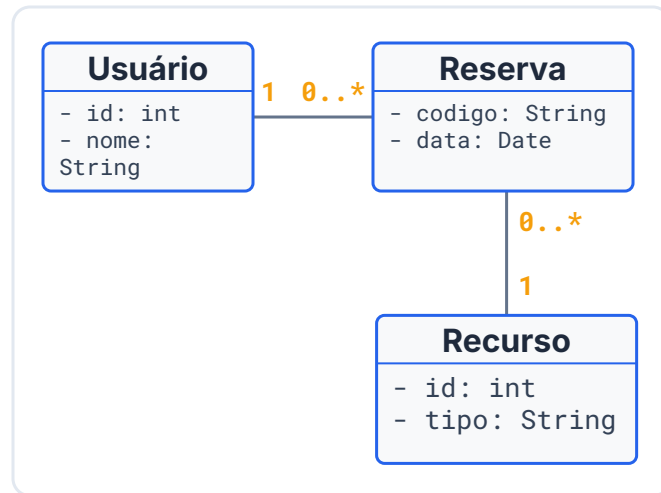
Depois de representar quem quer fazer o quê, que outras perguntas estruturais e dinâmicas ainda precisamos responder sobre o software para projetá-lo?

Diagrama de Classes: Perspectiva Estrutural

Quais são os principais elementos estruturais e como se relacionam?

Conceitos Essenciais:

- > **Classe:** Molde para objetos (ex: Usuário, Reserva, Recurso).
- > **Atributo:** Propriedades armazenadas pela classe.
- > **Associação:** Conexão semântica entre classes.
- > **Multiplicidade:** Define quantas instâncias se relacionam (**1**, **0..1**, *****, **1..***).



Importante: A multiplicidade não é uma decisão técnica. Ela vem diretamente das regras de negócio do domínio.

Do Caso de Uso ao Modelo de Classes

VISÃO COMPORTAMENTAL (PERSPECTIVA EXTERNA)



Aluno

Solicitar reserva

Transição:

Aluno → [Solicitar reserva]

Mudar de Perspectiva ↓

MODELO DE CLASSES ESTRUTURAL (PERSPECTIVA INTERNA)



Regra de Negócio Explicitada: Um **Usuário** pode possuir várias reservas (0..*), mas cada **Reserva** aloca exatamente um **Recurso** (1).

Exercício Rápido

Cenário: O sistema agora atende professores fazendo reservas de salas que contêm equipamentos específicos.

Tarefas:

- 1 Identificar classes candidatas.
- 2 Definir associações.
- 3 Estabelecer multiplicidades.

Ponto de Reflexão

Equipamento precisa obrigatoriamente ser uma classe, ou poderia ser apenas um atributo descritivo dentro de Sala?

Opção A: Classe

Equipamento

- tombamento: int
- tipo: String

Opção B: Atributo

Sala

- numero: int
- equip: String[]

Modelos Estruturais × Comportamentais



Visão Estrutural (Estática)

Representa elementos e relacionamentos estáveis do sistema.

Foco Principal

Entender **o que existe** no sistema.

Exemplo: Diagrama de Classes.



Visão Comportamental (Dinâmica)

Representa interações, eventos e fluxos ao longo do tempo.

Foco Principal

Entender **o que acontece** no sistema.

Exemplo: Diagrama de Sequência e Atividades.

Conclusão: Ambas as perspectivas são essenciais para projetar a solução completa.

Diagrama de Atividades

Elementos Fundamentais

- ○ **Nodos:** Inicial (cheio) e Final (com borda).
- ▭ **Ação:** Retângulo arredondado representando um passo.
- ◇ **Decisão e Merge:** Losangos para ramificar e unir fluxos.
- **Fork e Join:** Barras para dividir e sincronizar paralelos.

Diferença-Chave:

- Sequência:** ordem de mensagens entre objetos.
- Atividades:** fluxo lógico do processo.

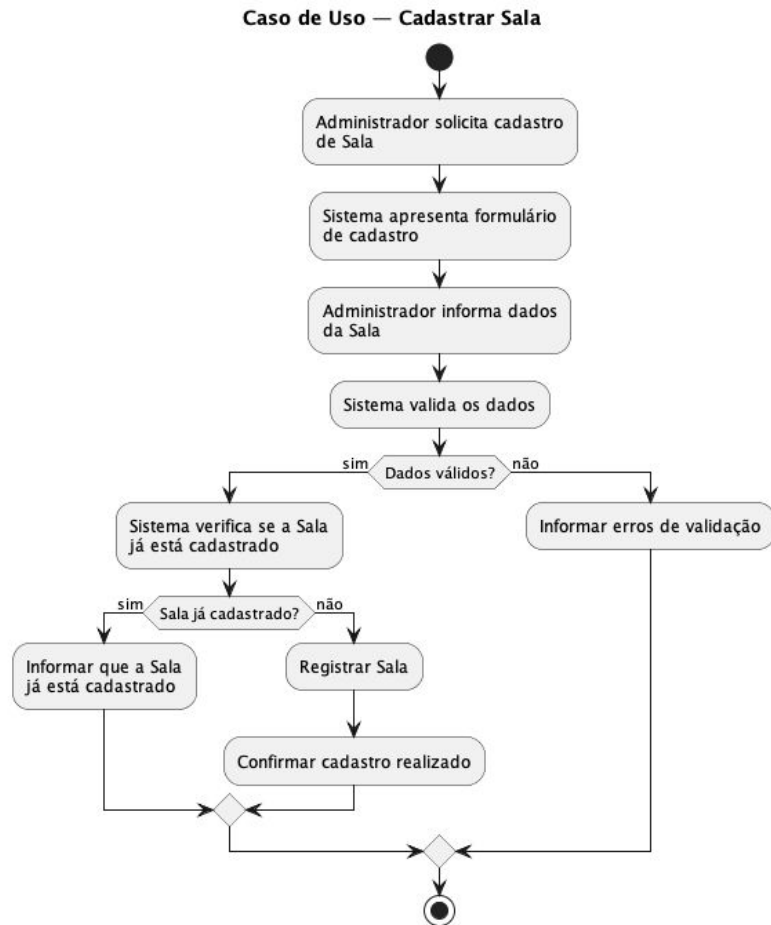


Diagrama de Atividades

Elementos Fundamentais

- ◎ **Nodos:** Inicial (cheio) e Final (com borda).
- ▭ **Ação:** Retângulo arredondado representando um passo.
- ◇ **Decisão e Merge:** Losangos para ramificar e unir fluxos.
- **Fork e Join:** Barras para dividir e sincronizar paralelos.

Diferença-Chave:

- Sequência:** ordem de mensagens entre objetos.
- Atividades:** fluxo lógico do processo.

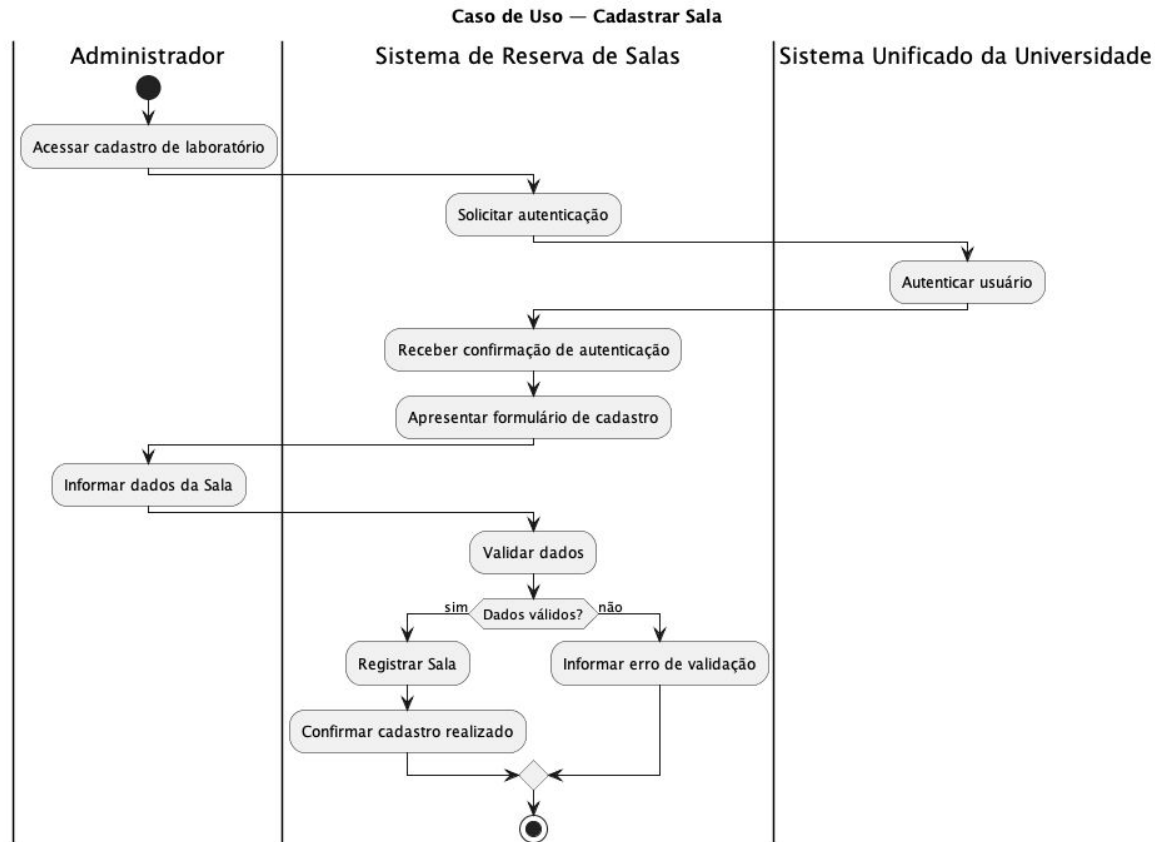


Diagrama de Sequência

Elementos da Notação

Participante

Entidades: Aluno, Interface, ServicoReserva.

Linha de vida: Eixo vertical representando o tempo.

Mensagem: Setas indicando chamadas ou retornos.

Ordem temporal:

Lida estritamente de cima para baixo.

A cronologia avança conforme a descida no eixo vertical.

Caso de Uso — Cadastrar Sala

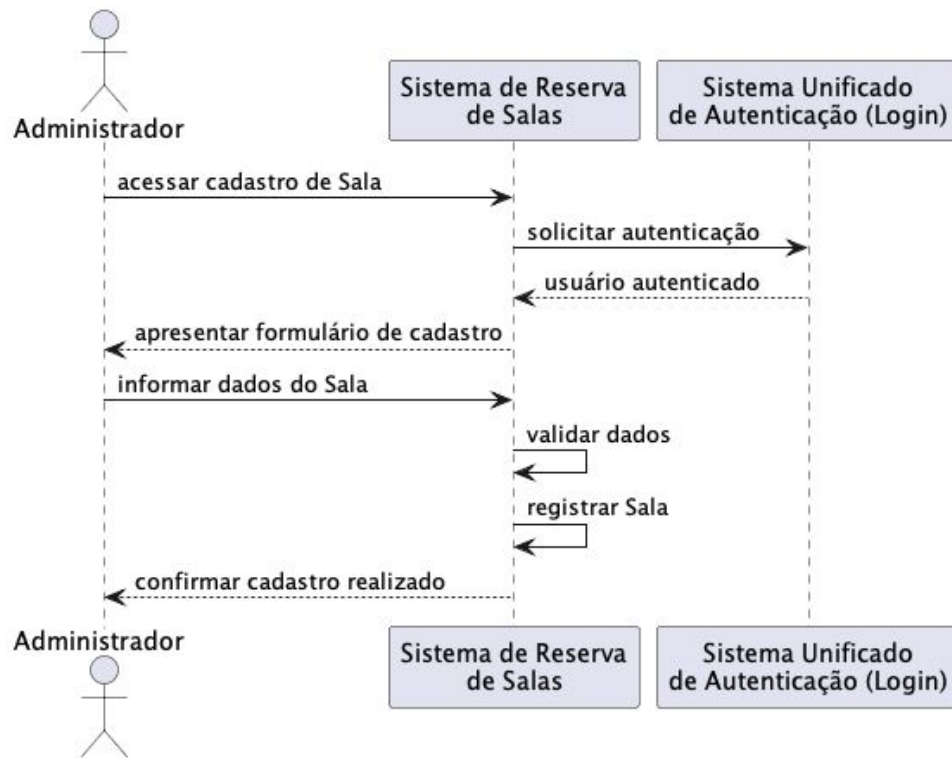


Diagrama de Sequência

Elementos da Notação

Participante

Entidades: Aluno, Interface, ServicoReserva.

Linha de vida: Eixo vertical representando o tempo.

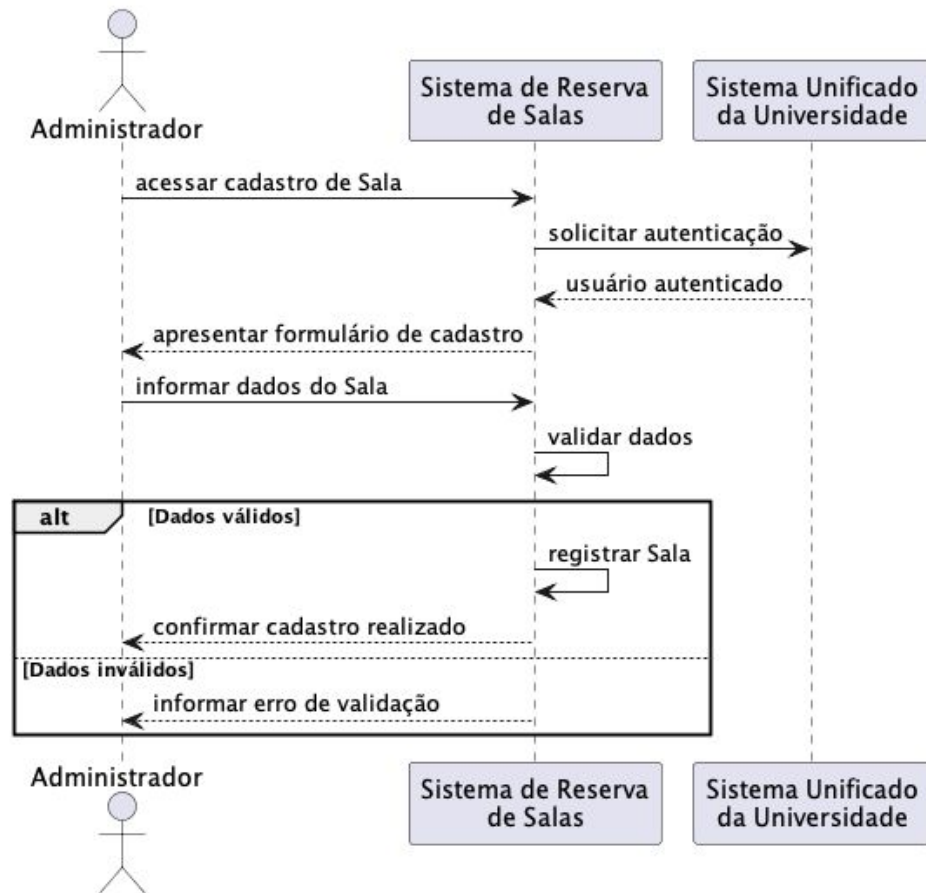
Mensagem: Setas indicando chamadas ou retornos.

Ordem temporal:

Lida estritamente de cima para baixo.

A cronologia avança conforme a descida no eixo vertical.

Caso de Uso — Cadastrar Sala



Atividade Rápida: Qual modelo escolher?

Situação A

Preciso mapear atores e seus objetivos externos.

>

Situação B

Preciso definir relacionamentos e multiplicidades de dados.

>

Situação C

Preciso visualizar a ordem temporal de mensagens entre componentes.

>

Situação D

Preciso desenhar um fluxo de processo com aprovação e paralelismo.

>

- Foco: Qual pergunta você está tentando responder?

Atividade Rápida: Qual modelo escolher?

Situação A

Preciso mapear atores e seus objetivos externos.

> Casos de Uso

Situação B

Preciso definir relacionamentos e multiplicidades de dados.

> Classes

Situação C

Preciso visualizar a ordem temporal de mensagens entre componentes.

> Sequência

Situação D

Preciso desenhar um fluxo de processo com aprovação e paralelismo.

> Atividades

● Foco: Qual pergunta você está tentando responder?

Diagrama de Pacotes

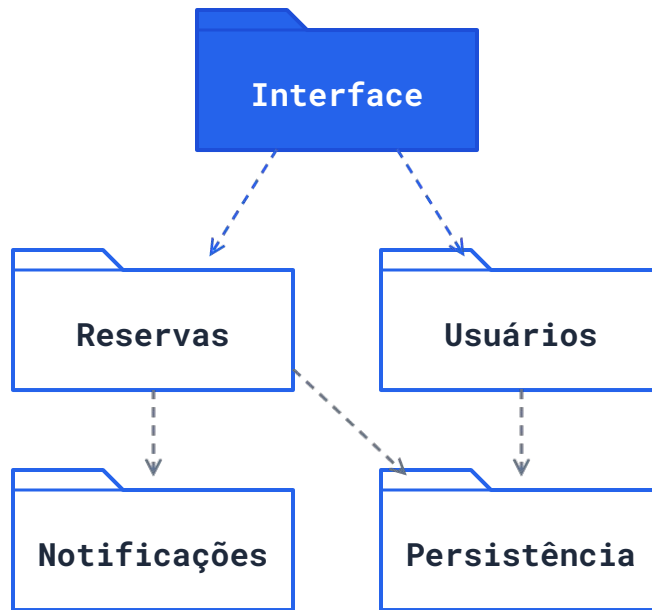
O diagrama de pacotes abstrai detalhes granulares de classes para fornecer uma **visão arquitetural ampla**.

Pacotes do Sistema de Reservas:

- **Interface** → Reservas, Usuários
- **Reservas** → Persistência, Notificações
- **Usuários** → Persistência
- **Notificações**
- **Persistência**

Essa visão de alto nível ajuda a organizar o código e gerenciar o acoplamento do sistema.

VISUALIZAÇÃO DE ARQUITETURA DE PACOTES



Extra: Diagrama de Fluxo de Dados (DFD)

DFD representa o fluxo e a transformação dos dados em um sistema.

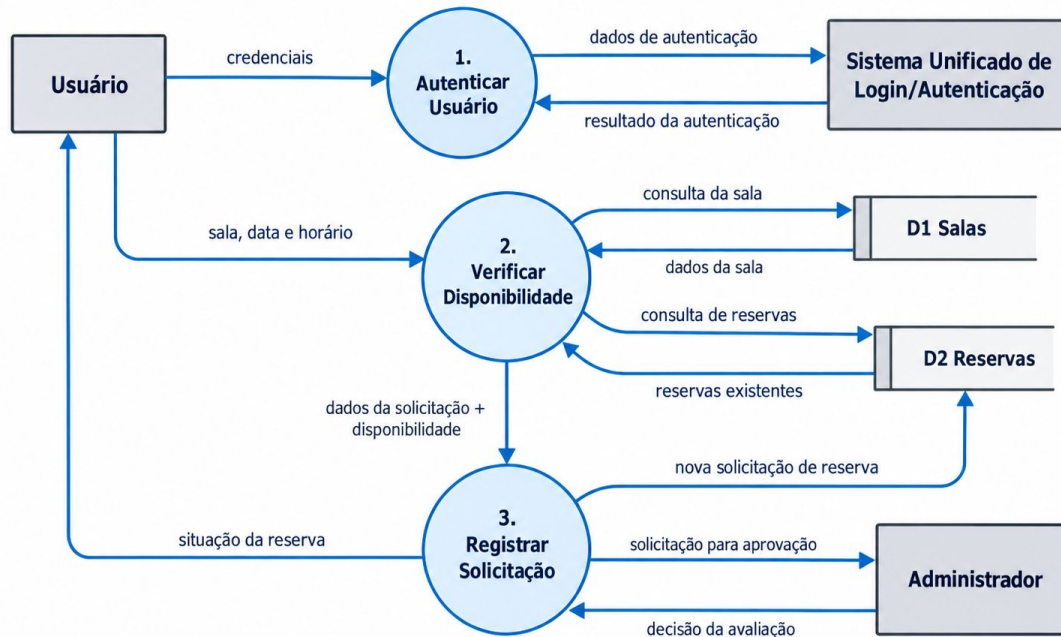
-  **Entidade externa:** Origem/destino de dados. Ex: Usuário
-  **Processo:** Transforma entrada em saída. Ex: Verificar
-  **Depósito de dados:** Armazena dados. Ex: Reservas
-  **Fluxo de dados:** Seta rotulada com dados. Ex: data/horário

DFD	Diagrama de Atividades
• Quais dados circulam? • Fluxos de dados • Processos e depósitos	• Qual caminho o fluxo segue? • Fluxos de controle • Ações e decisões

DFD mostra fluxo de dados, não fluxo de controle. Não confundir com Diagrama de Atividades ou fluxograma.
DFD não faz parte da UML.

DFD — Solicitar Reserva

Diagrama de Fluxo de Dados (nível 1)



Modelagem hoje: IA gera diagramas, mas quem valida?

O uso de Modelos de Linguagem (LLMs) está transformando a criação de diagramas.

~60%

Redução no tempo de geração

Achados Recentes (Eisenreich, Friedlaender & Wagner):

Geração de modelos de casos de uso com manutenção da qualidade estrutural.

Limitações:

- Risco de alucinações e inconsistências.
- Necessidade estrita de revisão humana.

Pergunta: Se a IA pode gerar diagramas em segundos, por que ainda aprender modelagem?

Perguntas para Reflexão

1 Qual o principal risco de aceitar um diagrama gerado por IA sem revisão?

2 Como a redução de ~60% no tempo altera o papel do engenheiro de software?

3 Cite um exemplo de inconsistência que a IA pode gerar em um caso de uso.

Modelagem hoje: IA gera diagramas, mas quem valida?

O uso de Modelos de Linguagem (LLMs) está transformando a criação de diagramas.

~60%

Redução no tempo de geração

Achados Recentes (Eisenreich, Friedlaender & Wagner):

Geração de modelos de casos de uso com manutenção da qualidade estrutural.

Limitações:

- Risco de alucinações e inconsistências.
- Necessidade estrita de revisão humana.

Pergunta: Se a IA pode gerar diagramas em segundos, por que ainda aprender modelagem?

Resposta: Porque gerar não é validar.

O pensamento crítico é essencial para garantir que o modelo atenda aos requisitos reais.

Perguntas para Reflexão

1 Qual o principal risco de aceitar um diagrama gerado por IA sem revisão?

2 Como a redução de ~60% no tempo altera o papel do engenheiro de software?

3 Cite um exemplo de inconsistência que a IA pode gerar em um caso de uso.

Conclusão

Redução de Complexidade

Modelos ajudam a **compreender, comunicar e discutir** a solução antes e durante a implementação. Em métodos ágeis, a UML geralmente é usada como **sketch**: o foco é representar apenas o relevante.

Visões Complementares

Diferentes diagramas respondem a **perguntas distintas** sobre o sistema. Por exemplo, **Casos de Uso** revelam quais atores interagem com o software e quais são seus objetivos principais.

Validação Humana

A IA acelera as etapas de modelagem (como criação de diagramas) em ~60%, mas cabe ao engenheiro validar alucinações e garantir o alinhamento com os requisitos reais.

Abstração & Estrutura

Diagramas de **Pacotes** e **Classes** permitem organizar a arquitetura de alto nível e as regras de negócio cruciais antes de escrever qualquer código.

Dinâmica & Tempo

Diagramas de **Sequência** e **Atividades** garantem a ordem cronológica correta das trocas de mensagens e o fluxo lógico dos processos.

Revisão humana segue indispensável para garantir coerência, completude e aderência aos requisitos.

PlantUML: Diagramas via Texto

Modelagem simplificada e versionável como código

Formas de Utilização

- [Editor Online](#) para testes rápidos
- [Extensão no VSCode](#) para desenvolvimento

Visão Geral

Criação de diagramas (UML e outros) a partir de texto simples.

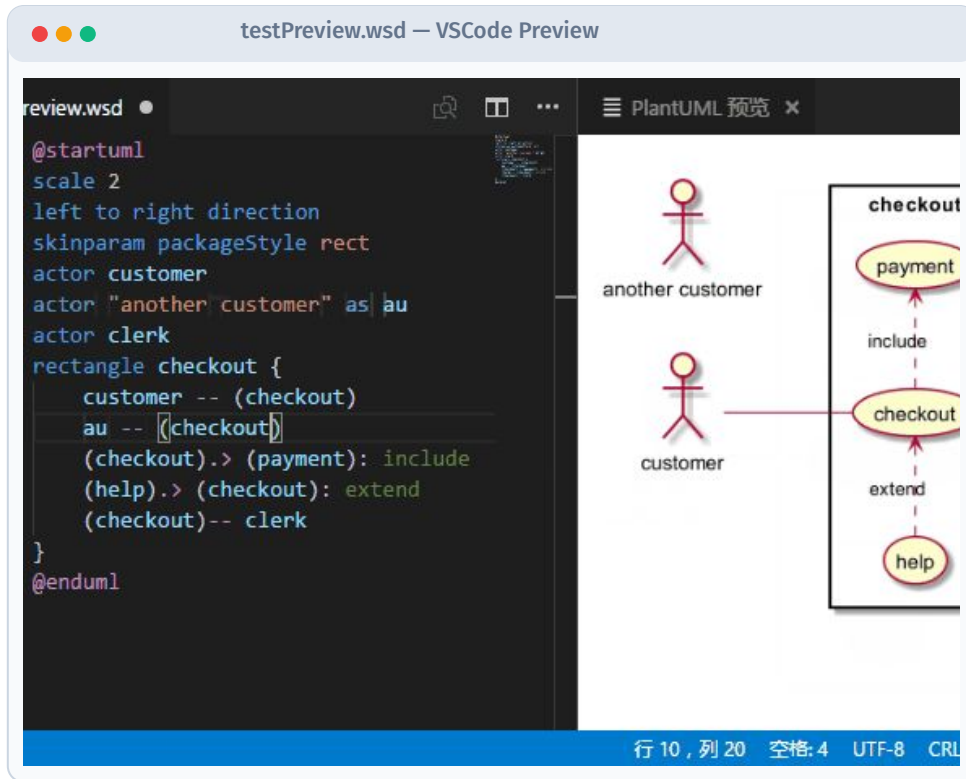
Documentação & Links

Site Oficial:

<https://plantuml.com/>

Guia Completo:

<https://plantuml.com/guide>



Atividade Prática: Modelagem de Software

Transformação de requisitos textuais em modelos visuais com PlantUML

1. Entregáveis do Lab

Transformar uma descrição textual de requisitos em dois modelos UML complementares:

Diagrama de Atividades

Foco na perspectiva do **fluxo do processo**.

Diagrama de Sequência

Foco nas **interações entre participantes** ao longo do tempo.

2. Foco da Atividade

20% PlantUML

Aprender a representar modelos via linguagem textual simples.

80% Modelagem & Análise

Interpretar requisitos, escolher abstrações, revisar sugestões do LLM e justificar decisões.

Modelos ajudam a representar aspectos relevantes (Pressman & Maxim) e diferentes perspectivas (Sommerville) do sistema.

3. Importante

Sem Orientação a Objetos

Não é necessário conhecer OO. Não será exigido conhecimento de classes, objetos ou herança.

Participantes da Interação

No Diagrama de Sequência, considere apenas os atores/agentes:

- **Estudante**
- **Sistema**
- **Administrador**
- **Serviço de Notificação**

Referências

Bibliografia Básica da Disciplina

Roger S. Pressman & Bruce R. Maxim

*Engenharia de Software: Uma Abordagem
Profissional*

Capítulos 8 e 11 (completo)

Ian Sommerville

*Engenharia de Software
Capítulo 5 (completo)*

Referências Complementares de Apoio

**From Requirements to Code: Understanding Developer
Practices in LLM-Assisted Software Engineering**

<https://ieeexplore.ieee.org/abstract/document/11190385>

**What Should We Engineer in Prompts? Training
Humans in Requirement-Driven LLM Use**

<https://dl.acm.org/doi/full/10.1145/3731756>

Dúvidas e Discussão